Week 1, Day 3: Collections, Comparators, Immutability and Hashing

Prepared for Upendra Kumar Sharma | 3–4 hours | Main topic: Java Collections deep dive | DSA: Hashing

Today's Schedule

- **0:00–0:50** Study the collection classes and their complexities (sections 1 and 2).
- **0:50–1:20** Study fail-fast iterators and Comparable vs Comparator (sections 3 and 4).
- **1:20–1:40** Study immutability and write the immutable class from section 5 yourself.
- **1:40–2:40** DSA: solve the 3 hashing problems (section 6). Spend at most 25 minutes per problem before reading the hint.
- **2:40–3:10** Answer the interview questions aloud (section 7) and add 10 flashcards.

1. The Collection Framework at a Glance

List

- **ArrayList:** backed by a resizable array, grows by about 1.5x. get(i) is O(1), add at the end is amortized O(1), insert or remove in the middle is O(n). Best default choice.
- **LinkedList:** doubly linked list. Add or remove at the ends is O(1), get(i) is O(n). It also implements Deque. In practice ArrayList or ArrayDeque is almost always faster because of cache locality.
- **Vector / Stack:** legacy and synchronized. Avoid them. Use ArrayDeque as a stack.
- **CopyOnWriteArrayList:** every write copies the whole array. Good for read-heavy lists with rare writes, such as listener lists.

Set

- **HashSet:** backed by a HashMap, O(1) average, no order.
- **LinkedHashSet:** keeps insertion order.
- **TreeSet:** red-black tree, sorted, O(log n), uses compareTo or a Comparator for equality.

Map

- **HashMap:** O(1) average, no order (internals from Day 1).
- **LinkedHashMap:** insertion or access order. With accessOrder=true and removeEldestEntry overridden, it becomes an LRU cache.

- **TreeMap:** sorted keys, O(log n), supports firstKey, floorKey, ceilingKey and range views.
- **ConcurrentHashMap:** thread-safe map (Day 1).

Queue and Deque

- **ArrayDeque:** fast stack and queue, no nulls. Preferred over Stack and LinkedList.
- **PriorityQueue:** binary heap. offer and poll are O(log n), peek is O(1). Iteration order is not sorted.
- **BlockingQueue** (LinkedBlockingQueue, ArrayBlockingQueue): used in producer-consumer designs and thread pools.

2. Choosing the Right Collection

- Need fast lookup by key: HashMap.
- Need sorted keys or range queries: TreeMap.
- Need insertion order plus fast lookup: LinkedHashMap.
- Need to remove duplicates: HashSet.
- Need a stack or queue: ArrayDeque.
- Need the top K or smallest N elements: PriorityQueue.
- Need thread-safe reads with rare writes: CopyOnWriteArrayList. Thread-safe map: ConcurrentHashMap.

Immutable and fixed-size collections

- **Arrays.asList:** fixed size, backed by the array. set() works, add() and remove() throw UnsupportedOperationException.
- **List.of / Set.of / Map.of (Java 9+):** truly immutable, null elements not allowed.
- **Collections.unmodifiableList:** read-only view of the original. If the original changes, the view changes too.
- **List.copyOf (Java 10+):** immutable copy.

3. Fail-Fast vs Fail-Safe Iterators

- **Fail-fast:** ArrayList, HashMap and similar classes keep a modCount counter. If the collection is structurally modified during iteration by anything other than the iterator itself, the next call throws ConcurrentModificationException. This is best-effort detection, not a guarantee.
- **Safe ways to remove while iterating:** Iterator.remove(), or removeIf(predicate), or collect into a new list.

- **Weakly consistent / snapshot iterators:** ConcurrentHashMap iterators never throw ConcurrentModificationException and may reflect some updates. CopyOnWriteArrayList iterators work on a snapshot taken at creation. People call these "fail-safe", but that is an informal term, so describe the exact behavior in interviews.

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4));
// Wrong: throws ConcurrentModificationException
// for (Integer x : list) if (x % 2 == 0) list.remove(x);

// Right
list.removeIf(x -> x % 2 == 0);
```

4. Comparable vs Comparator

- **Comparable:** the class defines its own natural ordering by implementing compareTo(T). One ordering per class.
- **Comparator:** a separate object that defines an ordering with compare(a, b). You can have many (by name, by salary, by date) and you do not need to change the class.
- **Contract:** the ordering should be consistent with equals. If compareTo returns 0 for two objects that are not equal, TreeSet and TreeMap will treat them as duplicates.
- **Pitfall:** do not compare ints with subtraction (a - b), because it can overflow. Use Integer.compare(a, b).

```
record Employee(String name, String dept, int salary) {}

List<Employee> sorted = employees.stream()
    .sorted(Comparator.comparing(Employee::dept)
            .thenComparing(Employee::salary, Comparator.reverseOrder())
            .thenComparing(Employee::name))
    .toList();
```

5. Immutability and Writing an Immutable Class

Rules for an immutable class

1. Declare the class final (or make the constructor private) so it cannot be subclassed.
2. Make all fields private and final.
3. Provide no setters.
4. Make defensive copies of mutable inputs in the constructor.
5. Return copies or unmodifiable views of mutable fields from getters.

6. Do not let this escape from the constructor.

```
public final class Order {
    private final String id;
    private final List<String> items;
    private final Date createdAt;

    public Order(String id, List<String> items, Date createdAt) {
        this.id = id;
        this.items = List.copyOf(items);          // defensive copy
        this.createdAt = new Date(createdAt.getTime());
    }
    public String getId() { return id; }
    public List<String> getItems() { return items; }   // already immutable
    public Date getCreatedAt() { return new Date(createdAt.getTime()); }
}
```

Why immutability matters

- Immutable objects are automatically thread-safe and need no synchronization.
- They are safe as HashMap keys because their hashCode cannot change.
- They can be shared and cached freely.
- Java records (Java 16+) give you an immutable data carrier with equals, hashCode and toString generated, but components that are mutable (a List, for example) still need defensive copies.

Why String is immutable

- The string pool can share instances safely.
- hashCode is cached, making Strings fast map keys.
- It protects security-sensitive values such as class names, file paths and connection URLs from being changed after validation.
- It is inherently thread-safe.

6. DSA: Hashing (3 Problems)

Problem 1: Top K Frequent Elements

- **Idea (heap):** count frequencies in a HashMap, then keep a min-heap of size k ordered by frequency. Pop the smallest whenever the heap exceeds k.
- **Idea (bucket sort):** create buckets indexed by frequency (0 to n), put each number in its frequency bucket, and read buckets from the highest down until you have k numbers.

- **Complexity:** heap O(n log k), bucket sort O(n).
- **Edge cases:** k equals the number of distinct elements, all elements identical, negative numbers.

Problem 2: Subarray Sum Equals K

- **Idea:** prefix sums with a HashMap that counts how many times each prefix sum has occurred. At each position, the number of subarrays ending here with sum k equals the count of prefix sums equal to (currentSum - k).
- **Start the map with {0: 1}** so subarrays beginning at index 0 are counted.
- **Complexity:** O(n) time, O(n) space.
- **Why not a sliding window:** the array can contain negative numbers, so a window cannot be shrunk by a simple rule.

```
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1);
    int sum = 0, result = 0;
    for (int x : nums) {
        sum += x;
        result += prefixCount.getOrDefault(sum - k, 0);
        prefixCount.merge(sum, 1, Integer::sum);
    }
    return result;
}
```

Problem 3: Longest Consecutive Sequence

- **Idea:** put all numbers in a HashSet. Only start counting from a number whose predecessor (num - 1) is not in the set, so each sequence is walked exactly once.
- **Complexity:** O(n) time, O(n) space. Sorting would be O(n log n), which does not meet the required bound.
- **Edge cases:** empty array, duplicates, negative numbers.

```
public int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int x : nums) set.add(x);
    int best = 0;
    for (int x : set) {
        if (!set.contains(x - 1)) {
            int len = 1;
            while (set.contains(x + len)) len++;
            best = Math.max(best, len);
        }
    }
    return best;
}
```

7. Interview Questions to Answer Aloud

1. ArrayList vs LinkedList: complexities, and which one you would use and why.
2. How does ArrayList grow? What is the cost of add at the end?
3. HashSet vs TreeSet vs LinkedHashSet.
4. What is ConcurrentModificationException, and how do you avoid it?
5. Fail-fast vs weakly consistent iterators: give one example of each.
6. Comparable vs Comparator. When do you use each?
7. What happens in a TreeSet if compareTo returns 0 for two different objects?
8. How would you sort employees by department, then by salary descending, then by name?
9. What is the difference between Arrays.asList, List.of and Collections.unmodifiableList?
10. How do you create an immutable class? Why do you need defensive copies?
11. Why is String immutable?
12. How would you implement an LRU cache using LinkedHashMap?
13. When is a PriorityQueue the right choice? Is its iteration order sorted?

8. End-of-Day Checklist

- [ ] I can state the time complexity of the main operations of ArrayList, LinkedList, HashMap, TreeMap and PriorityQueue.
- [ ] I can explain fail-fast iterators and the safe ways to remove during iteration.
- [ ] I can sort a list by multiple fields with Comparator.comparing and thenComparing.

- [ ] I can write an immutable class with defensive copies from memory.
- [ ] I solved all 3 hashing problems and can explain the prefix-sum trick.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Day 4): Multithreading basics (thread lifecycle, synchronized, volatile, happens-before), plus two-pointer problems.